

DAY 1 — COMPLETE STUDY NOTES

Pointers · Arrays · Dynamic Memory · Function Pointers · Callbacks · Complex Declarations

#	Topic	Page
1	Pointer Fundamentals & Arithmetic	—
2	1D · 2D · 3D Arrays in Depth	—
3	Pointer–Array Duality	—
4	Dangling · Void · NULL · Wild Pointers	—
5	Dynamic Memory Allocation (new/delete, malloc/free)	—
6	Complex Declarations — Reading Rules (clockwise spiral)	—
7	Function Pointers	—
8	Callbacks in C/C++	—
9	5 GATE PYQ MCQs — Fully Solved	—
10	Quick-Reference Cheat Sheet	—

Strategy: *Concept* → *Trace* → *Code* → *Build*. Every topic links directly to the search engine project.

■ SECTION 1 — POINTER FUNDAMENTALS & ARITHMETIC ■

A **pointer** is a variable whose value is the memory address of another variable. Pointers are the foundation of systems programming: they enable dynamic memory allocation, efficient array traversal, function callbacks, and OS-level interactions.

1.1 What is a Pointer?

Every variable lives at some address in RAM. A pointer stores that address. The size of any pointer on a 64-bit system is **8 bytes** — regardless of what it points to.

```
int x = 42;

int *p = &x; // p holds the ADDRESS of x

printf("%d\n", x); // 42 – value of x
printf("%p\n", p); // 0x7ffd... – address stored in p
printf("%d\n", *p); // 42 – dereference: value AT that address
printf("%p\n", &p); // address of the pointer variable itself
```

Variable	Address	Value stored
x	0x1000	42
p	0x1008	0x1000 (address of x)
*p	–	42 (same as x – dereference)

1.2 Pointer Arithmetic

When you add/subtract an integer from a pointer, the compiler scales by the **size of the pointed-to type**. On most 64-bit Linux systems: int = 4 B, double = 8 B, char = 1 B.

```
int arr[] = {10, 20, 30, 40, 50};

int *p = arr; // points to arr[0]

p++; // now points to arr[1] (+4 bytes on 32-bit, same on 64-bit)
p += 2; // now points to arr[3] (+8 bytes)

// Pointer subtraction gives ELEMENT count, not byte count
int *q = &arr[4];
ptrdiff_t diff = q - p; // diff = 1 (arr[4] - arr[3])

// Array traversal using ONLY pointer arithmetic (no index variable)
for (int *it = arr; it != arr + 5; ++it)
    printf("%d ", *it); // 10 20 30 40 50
```

Key rule: `p + n` moves `n * sizeof(*p)` bytes forward. This is why pointer arithmetic is type-aware.

1.3 Pointer vs Reference (C++)

Feature	Pointer (T*)	Reference (T&)
Reassignable	Yes — can point to anything	No — bound at init, forever
Nullable	Yes — nullptr is valid	No — must bind to real object
Arithmetic allowed	Yes	No

Syntax to access	*ptr or ptr->member	ref.member (transparent)
Use case	Optional ownership, arrays, DMA	Aliasing, out-params, for-each
Assembly overhead	Load address then deref	Same as pointer (optimised away)

1.4 NULL vs nullptr vs void*

```
// NULL (C style) – macro, expands to (void*)0 or 0
```

```
int *p1 = NULL;
```

```
// nullptr (C++11+) – keyword, type-safe null pointer constant
```

```
int *p2 = nullptr; // PREFER this in modern C++
```

```
// void* – generic pointer, can hold any pointer type
```

```
void *vp = malloc(16); // malloc returns void*
```

```
int *ip = (int*)vp; // must cast before dereference
```

```
// CANNOT do: *vp = 5; (no size info, compiler error)
```

```
// CANNOT do: vp++; (pointer arithmetic illegal on void*)
```

nullptr vs NULL: NULL can silently convert to int (causing overload resolution bugs). nullptr has type std::nullptr_t and only converts to pointer types.

2.1 1D Arrays

A 1D array is a contiguous block of memory holding N elements of the same type. The array name is a *non-modifiable* pointer to its first element. There is no bounds checking — you are responsible.

```
int a[5] = {10, 20, 30, 40, 50};

// a == &a[0] (address of first element)
// a + i == &a[i] (address of i-th element)
// *(a+i) == a[i] (two syntaxes are IDENTICAL after compilation)

// Reverse using pointer arithmetic ONLY (Day 1 build task):
int *lo = a, *hi = a + 4;
while (lo < hi) {
    int tmp = *lo;
    *lo++ = *hi;
    *hi-- = tmp;
}

// Result: {50, 40, 30, 20, 10}
```

Memory layout (int a[5], each int = 4 bytes):

Index	a[0]	a[1]	a[2]	a[3]	a[4]
Address	0x1000	0x1004	0x1008	0x100C	0x1010
Value	10	20	30	40	50

2.2 2D Arrays — Row-Major Layout

A 2D array `int a[R][C]` is stored in **row-major order** in C/C++: all elements of row 0, then row 1, etc. Total size = $R \times C \times \text{sizeof(int)}$.

```
int a[3][4]; // 3 rows, 4 cols = 12 ints = 48 bytes

// Element access formula:
// a[i][j] == *(a[0] + i*4 + j) == *( (int*)a + i*C + j )

// Pointer to a ROW (type: int (*)[4])
int (*row_ptr)[4] = a; // row_ptr points to row 0
row_ptr++; // now points to row 1 (+16 bytes)

// Pass 2D array to function — MUST specify column count:
void print(int arr[][4], int rows) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < 4; j++)
            printf("%d ", arr[i][j]);
}
```

2D Memory Layout (3×4 ints, base address = 0x2000):

	Col 0	Col 1	Col 2	Col 3
Row 0	0x2000	0x2004	0x2008	0x200C

Row 1	0x2010	0x2014	0x2018	0x201C
Row 2	0x2020	0x2024	0x2028	0x202C

GATE trap: `sizeof(a) = 48 bytes`. `sizeof(a[0]) = 16 bytes (one row)`. `sizeof(a[0][0]) = 4 bytes`.

```
// Traverse 2D array as flat 1D (pointer trick):
int *flat = (int*)a;
for (int k = 0; k < 3*4; k++)
printf("%d ", flat[k]); // safe, all contiguous memory
```

2.3 3D Arrays

A 3D array `int a[D][R][C]` extends the pattern: D 'planes', each plane has R rows, each row has C ints. Address of element `a[d][r][c] = base + (d*R*C + r*C + c) * sizeof(int)`.

```
int a[2][3][4]; // 2 planes × 3 rows × 4 cols = 24 ints = 96 bytes
```

```
// General element address (C = 4, R = 3):
// &a[d][r][c] = (int*)a + d*(3*4) + r*4 + c
// = (int*)a + d*12 + r*4 + c
```

```
// sizeof expressions:
// sizeof(a) = 96 (entire array)
// sizeof(a[0]) = 48 (one plane = 3×4 ints)
// sizeof(a[0][0]) = 16 (one row = 4 ints)
// sizeof(a[0][0][0]) = 4 (one int)
```

```
// Pointer to a 2D plane:
```

```
int (*plane)[3][4] = a; // points to a[0]
plane++; // points to a[1] (+48 bytes)
```

General n-D address formula: For `a[i][j][k]` in `a[D][R][C]`: *offset = i*(R*C) + j*C + k*, multiply by *sizeof(element)*.

■ SECTION 3 — POINTER-ARRAY DUALITY ■

Arrays and pointers are deeply intertwined in C/C++ but they are **NOT** the same thing. Understanding the differences prevents a class of GATE trap questions.

Expression	Type	Meaning
<code>a</code>	<code>int*</code> (decays)	Address of <code>a[0]</code> — NOT a pointer variable
<code>&a</code>	<code>int (*)[5]</code>	Address of entire array — NOT same as <code>a</code>
<code>a + 1</code>	<code>int*</code>	Address of <code>a[1]</code>
<code>&a + 1</code>	<code>int (*)[5]</code>	Address PAST the entire array (+20 bytes for <code>int[5]</code>)
<code>*a</code>	<code>int</code>	Value at <code>a[0]</code> $\equiv$ <code>a[0]</code>
<code>*(a + i)</code>	<code>int</code>	Value at <code>a[i]</code> $\equiv$ <code>a[i]</code>
<code>int *p = a</code>	<code>int*</code>	<code>p</code> is a pointer variable; <code>a</code> is not
<code>sizeof(a)</code>	<code>size_t</code>	20 bytes (if <code>int a[5]</code>) — full array size
<code>sizeof(p)</code>	<code>size_t</code>	8 bytes — size of pointer, NOT array!

GATE trap — `&a` vs `a`: If `int a[5]` starts at `0x1000`, then `a == 0x1000` and `&a == 0x1000` — same numeric value but different types! `a+1 == 0x1004` but `&a+1 == 0x1014` (+20 bytes).

```
int a[5];
int *p = a; // p = 0x1000
int *q = a + 1; // q = 0x1004 (+4 bytes)

// &a has type int(*)[5]:
int (*ap)[5] = &a; // ap = 0x1000
int (*bp)[5] = &a + 1; // bp = 0x1014 (+20 bytes!)

// Double pointer trick for 2D arrays:
int b[3][4];
int (*rp)[4] = b; // pointer to array of 4 ints
rp++; // skips one ENTIRE ROW (16 bytes)
```

4.1 Dangling Pointers

A **dangling pointer** holds an address that is no longer valid — the memory it pointed to has been freed, the variable has gone out of scope, or the function that owned it has returned. Dereferencing a dangling pointer is **undefined behaviour**.

Cause 1 — Freed heap memory:

```
int *p = (int*)malloc(sizeof(int));
*p = 42;
free(p);
// p is NOW a dangling pointer
*p = 99; // UNDEFINED BEHAVIOUR – heap may be reused
// Fix: set to nullptr immediately after free
free(p);
p = nullptr;
```

Cause 2 — Returning address of local variable:

```
int* bad() {
    int x = 5; // x lives on the STACK
    return &x; // stack frame destroyed on return!
} // caller gets dangling pointer

// Fix: use static, or allocate on heap:
int* ok() {
    static int x = 5; // static: lifetime = entire program
    return &x;
}
```

Cause 3 — Variable goes out of scope:

```
int *p;
{
    int a = 10;
    p = &a;
} // a is DESTROYED here – p now dangling
printf("%d", *p); // garbage or crash
```

4.2 Void Pointer (void*)

A `void*` is a **generic pointer** — it can hold the address of any type. You cannot dereference or do arithmetic on it directly; you must cast first. Used by `malloc()`, `memcpy()`, and generic containers.

```
int x = 42;
float f = 3.14f;
void *vp;
vp = &x;
```

```
printf("%d\n", *(int*)vp); // cast then deref

vp = &f;

printf("%.2f\n", *(float*)vp); // cast then deref

// malloc returns void* – always cast in C++:
int *arr = (int*)malloc(10 * sizeof(int));

// ILLEGAL:
// *vp = 5; // error: cannot dereference void*
// vp++; // error: sizeof(void) is undefined
```

4.3 NULL Pointer

A **NULL pointer** explicitly holds 'no address' (typically 0 / nullptr). It is a *defined* value — unlike a dangling pointer (whose address is garbage). Always null-check before dereferencing.

```
int *p = nullptr; // C++11
int *q = NULL; // C (macro = 0)

if (p != nullptr) { // safe guard
printf("%d", *p);
}

// NULL pointer comparison is VALID – gives 0/false
// Dereferencing NULL is UNDEFINED (usually SIGSEGV crash)
```

4.4 Wild Pointer

A **wild pointer** is one that has *never been initialised*. It contains whatever garbage was in that stack slot. Wild pointers are worse than NULL pointers because they silently corrupt random memory.

```
int *p; // WILD pointer – not initialised, not NULL
*p = 42; // corrupts random memory – UNDEFINED BEHAVIOUR

// Fix: ALWAYS initialise pointers
int *p = nullptr; // safe default until you have a real address
int *q = new int; // or allocate immediately
```

Pointer Type	Address value	Safe to dereference?	How to fix
NULL	0 (defined)	NO — SIGSEGV	Check != nullptr first
Wild	Garbage (undefined)	NO — random corruption	Always initialise
Dangling	Was valid, now freed	NO — undefined behav.	Set = nullptr after free/scope exit
Void*	Any valid address	Only after cast	Cast to concrete type first

■ SECTION 5 — DYNAMIC MEMORY ALLOCATION ■

Stack memory is automatically managed but limited (~8MB). Heap memory is manually managed, larger, and persists until explicitly freed. The crawler will use the heap extensively for URL strings, HTML buffers, and data structures.

5.1 C-style: malloc / calloc / realloc / free

```
#include <stdlib.h>

// malloc: allocate n bytes, contents are UNINITIALISED
int *p = (int*)malloc(10 * sizeof(int));
if (p == NULL) { /* handle allocation failure */ }

// calloc: allocate and ZERO-INITIALISE
int *q = (int*)calloc(10, sizeof(int)); // all zeros

// realloc: resize existing allocation
p = (int*)realloc(p, 20 * sizeof(int)); // may move to new address!
// WARNING: if realloc fails it returns NULL but old memory still valid
// Always use a temp pointer:
int *tmp = (int*)realloc(p, 20 * sizeof(int));
if (tmp) p = tmp; // only update p on success

// free: release heap memory
free(p);

p = NULL; // good practice: avoid dangling pointer
```

NEVER free the same pointer twice (double-free). **NEVER** use memory after freeing it (use-after-free). Both are undefined behaviour and common security vulnerabilities.

5.2 C++ style: new / delete

```
// Single object
int *p = new int; // uninitialised
int *q = new int(42); // initialised to 42
delete p;
delete q;

// Arrays
int *arr = new int[10]; // uninitialised array
int *brr = new int[10](); // zero-initialised array
delete[] arr; // MUST use delete[], not delete
delete[] brr;

// new vs malloc:
// new calls constructor + throws std::bad_alloc on failure
// malloc doesn't call ctor + returns NULL on failure

// new (nothrow) – returns nullptr instead of throwing:
int *safe = new (std::nothrow) int[1000000];
if (!safe) { /* handle */ }
```

5.3 Dynamic 2D Arrays on Heap

```
// Method 1: Pointer to pointer (non-contiguous rows)

int **mat = new int*[R];

for (int i = 0; i < R; i++)
    mat[i] = new int[C];

// Access: mat[i][j]
// Free:

for (int i = 0; i < R; i++) delete[] mat[i];
delete[] mat;

// Method 2: Single flat allocation (contiguous – BETTER for cache)

int *flat = new int[R * C];

// Access: flat[i*C + j] – equivalent to mat[i][j]

delete[] flat;

// Method 3: Modern C++ (prefer this)
#include <vector>

std::vector<std::vector<int>> v(R, std::vector<int>(C, 0));

// Access: v[i][j] – automatic memory management
```

5.4 Memory Leak Example + Detection

```
// Intentional leak (for valgrind demo):

void leak() {
    int *p = new int[1000]; // never deleted
    // function returns – p lost – 4000 bytes leaked
}

// Compile and check:
// g++ -g -o prog prog.cpp
// valgrind --leak-check=full ./prog
// Output: == definitely lost: 4,000 bytes in 1 blocks

// AddressSanitizer (compile-time instrumentation):
// g++ -fsanitize=address -g -o prog prog.cpp
// Run ./prog – ASan prints exact leak site with line numbers
```

■ SECTION 6 — COMPLEX DECLARATIONS (CLOCKWISE SPIRAL RULE) ■

C declarations can nest arbitrarily: pointers to arrays, arrays of pointers, pointers to functions, arrays of function pointers, etc. The **clockwise spiral rule** (right-left rule) lets you read any declaration.

6.1 The Right-Left Rule (Reading Algorithm)

Step	Action
1	Find the identifier (variable/function name) — start here
2	Look RIGHT first: if you see [] → 'array of ...', () → 'function taking ...'
3	If you hit) or end of right, look LEFT: * → 'pointer to ...'
4	Keep alternating right/left until you exhaust the declaration
5	A parenthesised group like (*name) forces you to deal with * before []

`int *p`

Reading: `p` is a pointer to `int`

- start at `p`
- → nothing to the right
- ← `int*`
- = pointer to `int`

`int *a[5]`

Reading: `a` is an array of 5 pointers to `int`

- start at `a`
- → `[5]` = array of 5
- ← `*` = pointer to
- ← `int`
- = array of 5 pointers to `int`

`int (*a)[5]`

Reading: `a` is a pointer to an array of 5 ints

- start at `a`
- ← (inside parens) `*` = pointer to
- → `[5]` = array of 5
- ← `int`
- = pointer to array of 5 int

`int (*fp)(int, float)`

Reading: `fp` is a pointer to a function taking `(int, float)` and returning `int`

- start at `fp`
- ← `*` = pointer to
- → `(int, float)` = function taking ...
- ← `int` = returning `int`

`int *(*fp)(int *)`

Reading: fp is a pointer to a function taking pointer-to-int and returning pointer-to-int

- start at fp
- ← * = pointer to
- → (int*) = function taking pointer-to-int
- ← *int = returning pointer-to-int

`int (*a[5])(void)`

Reading: a is an array of 5 pointers to functions taking void and returning int

- start at a
- → [5] = array of 5
- ← * = pointer to
- → (void) = function taking void
- ← int = returning int

6.2 typedef Simplification Strategy

```
// Complex: pointer to function taking int, returning pointer-to-int
int *(*complicated)(int);

// Use typedef to decompose:
typedef int* IntPtr; // step 1: name the return type
typedef IntPtr (*FuncPtr)(int); // step 2: name the function pointer type
FuncPtr f; // now f is the same as complicated above

// Real-world example from thread pool callbacks:
typedef void (*TaskFunc)(void *arg);
TaskFunc callbacks[64]; // array of 64 task function pointers
```

f SECTION 7 — FUNCTION POINTERS IN C/C++ *f*

Every function compiled into a program lives at a fixed address in the code segment. A **function pointer** stores that address and lets you call the function indirectly. This is how `qsort`, callbacks, thread pool tasks, and virtual dispatch work.

7.1 Syntax and Declaration

```
// SYNTAX: return_type (*ptr_name)(param_types...);

int multiply(int a, int b) { return a * b; }
void greet(const char *name) { printf("Hello %s", name); }

// Declaring function pointers:
int (*fp1)(int, int); // points to function int f(int, int)
void (*fp2)(const char *); // points to function void f(const char*)

// Assigning – function name WITHOUT parentheses gives its address:
fp1 = multiply; // OR: fp1 = &multiply; (both are valid)
fp2 = greet;

// Calling – with OR without explicit deref:
int r1 = fp1(3, 4); // preferred – looks like normal call
int r2 = (*fp1)(3, 4); // explicit deref – also valid

// sizeof(fp1) == 8 bytes on 64-bit (just a pointer)
```

7.2 Passing Function Pointers as Parameters

```
// Higher-order function: receives a function, calls it
void apply(int a, int b, int (*op)(int, int)) {
    printf("Result: %d\n", op(a, b));
}

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

apply(10, 3, add); // prints 13
apply(10, 3, sub); // prints 7
apply(10, 3, [](int a, int b){ return a*b; }); // C++11 lambda

// qsort example – classic C callback pattern:
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int arr[] = {5, 2, 8, 1, 9};
qsort(arr, 5, sizeof(int), cmp);
```

7.3 Array of Function Pointers (Dispatch Table)

```
// Dispatch table – O(1) lookup vs O(n) if/else chain
typedef int (*BinaryOp)(int, int);
```

```

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }
int dvd(int a, int b) { return b ? a/b : 0; }

BinaryOp ops[4] = { add, sub, mul, dvd };

// ops[0](6,2) = 8, ops[1](6,2) = 4, etc.

// Used in crawler thread pool to dispatch task types:
// TaskFunc handlers[TASK_TYPE_COUNT];
// handlers[TASK_FETCH] = do_fetch;
// handlers[TASK_PARSE] = do_parse;
// handlers[task->type](task->arg);

```

7.4 Function Pointers vs std::function (C++)

```

#include <functional>

// std::function is more flexible – accepts lambdas, functors, methods:
std::function<int(int,int)> fp = [](int a, int b){ return a+b; };
fp(3, 4); // 7

// Raw function pointer: ZERO overhead, cannot capture lambdas
int (*raw)(int,int) = add; // fine – no closure

// When to use which:
// Raw fp: callbacks with no captures, C APIs, thread pools
// std::function: when you need closures (captures), flexibility
// Caveat: std::function has ~40ns overhead vs ~0ns for raw pointer

```

■ SECTION 8 — CALLBACKS IN C/C++ ■

A **callback** is a function passed as an argument to another function, to be executed ('called back') at a specific point. Callbacks are the foundation of event-driven systems, asynchronous I/O, and thread pools.

8.1 Basic Callback Pattern

```
// Step 1: Define callback function
void on_fetch_complete(const char *url, int status_code) {
    printf("Fetched %s → HTTP %d\n", url, status_code);
}

// Step 2: Function that accepts a callback
typedef void (*FetchCallback)(const char*, int);

void fetch_url(const char *url, FetchCallback cb) {
    // ... do network I/O ...
    int status = 200; // pretend
    cb(url, status); // invoke the callback
}

// Step 3: Wire it up
fetch_url("https://example.com", on_fetch_complete);
```

8.2 Callbacks with User Data (void* context pattern)

```
// The void* 'context' (or 'userdata') pattern lets callers
// attach arbitrary state without changing the callback signature.

typedef void (*TaskCallback)(void *ctx, int result);

struct CrawlTask {
    char url[256];
    int depth;
};

void crawl_done(void *ctx, int result) {
    CrawlTask *task = (CrawlTask*)ctx; // cast back
    printf("Done: %s depth=%d result=%d\n",
        task->url, task->depth, result);
}

void run_task(void *ctx, TaskCallback cb) {
    // ... do work ...
    cb(ctx, 200);
}

CrawlTask t = {"https://example.com", 2};
run_task(&t, crawl_done);
```

8.3 C++ Lambda as Callback

```

#include <functional>

// Modern C++: lambdas replace explicit callback structs
void fetch(const std::string &url,
std::function<void(int)> on_done) {
// ... fetch ...
on_done(200);
}

// Capturing context in lambda (no void* needed):
std::string my_url = "https://example.com";
int my_depth = 2;

fetch(my_url, [my_url, my_depth](int status) {
printf("Fetched %s (depth=%d) → %d\n",
my_url.c_str(), my_depth, status);
});

```

8.4 qsort — Classic C Callback in Standard Library

```

#include <stdlib.h>

// qsort signature:
// void qsort(void *base, size_t n, size_t size,
// int (*compar)(const void*, const void*));

// Comparator callback — must return <0, 0, or >0
int cmp_int_asc(const void *a, const void *b) {
return (*(int*)a) - (*(int*)b); // ascending
}

int cmp_int_desc(const void *a, const void *b) {
return (*(int*)b) - (*(int*)a); // descending
}

int arr[] = {5, 2, 8, 1, 9, 3};
qsort(arr, 6, sizeof(int), cmp_int_asc);
// arr is now: 1 2 3 5 8 9

// TRAP: cmp_int_asc(INT_MAX, INT_MIN) overflows! Safe version:
int safe_cmp(const void *a, const void *b) {
int x = *(int*)a, y = *(int*)b;
return (x > y) - (x < y); // no overflow
}

```


◆ SECTION 9 — 5 GATE PYQ MCQs — FULLY SOLVED ◆

These five questions are selected from GATE CS 1997–2014 and specifically target pointer arithmetic, arrays, dangling pointers, dynamic memory, and function pointers — all core Day 1 topics.

GATE CS 2000 | MCQ 1

Q1. Match the following code fragments with the most appropriate description:

X: `m = malloc(5); m = NULL;`

Y: `free(n); n->value = 5;`

Z: `char *p; *p = 'a';`

Options: 1 = using dangling pointers, 2 = using uninitialized (wild) pointers, 3 = memory leak

(A) X→1, Y→2, Z→3

(B) X→3, Y→2, Z→1

✓ (C) X→3, Y→1, Z→2

(D) X→1, Y→3, Z→2

Answer: (C) — X: `malloc` allocates heap memory but then the pointer `m` is immediately overwritten with `NULL`. The allocated block is now unreachable — this is a MEMORY LEAK (3). Y: `free(n)` releases `n`'s memory; `n->value=5` then accesses freed memory — DANGLING POINTER (1). Z: `char *p` is declared but never initialised — WILD pointer; writing through it is undefined (2). Answer: X→3, Y→1, Z→2.

GATE CS 2001 | MCQ 2

Q2. Consider the following three C functions:

P1: `int *g() { int x; return &x; }`

P2: `int *g() { int *px; *px = 10; return px; }`

P3: `int *g() { int *px = (int*)malloc(sizeof(int)); *px = 10; return px; }`

Which functions are LIKELY TO CAUSE PROBLEMS with pointers?

(A) Only P3

(B) Only P1 and P3

✓ (C) Only P1 and P2

(D) P1, P2, and P3

Answer: (C) — P1: `x` is a local variable on the stack. Returning `&x` is returning the address of a stack variable. After `g()` returns, `x` is destroyed — caller has a DANGLING pointer. PROBLEM. P2: `px` is an uninitialised pointer (WILD pointer). Writing `*px = 10` writes to a random memory location. PROBLEM. P3: `px` is allocated on the heap via `malloc`. Heap memory persists after function return. Caller receives a valid pointer. NOT a problem (though caller must free it). Answer: Only P1 and P2.

Q3. Consider the following C program. What is the output?

```
int main() {
    int *ptr, a, b, c;
    a=5; b=11; c=20;
    ptr=&a; *ptr=c;
    ptr=&c; a=*&b;
    c=*ptr-a;
    printf("%d", c);
}
```

✓ (A) 9

(B) 15

(C) 0

(D) 20

Answer: (A) — Trace step by step: a=5, b=11, c=20. ptr=&a; *ptr=c → a=20. Now a=20, b=11, c=20. ptr=&c; (ptr points to c). a=*&b; → a=b=11. Now a=11. c=*ptr-a → c = *ptr - a = 20 - 11 = 9. printf prints 9. Answer: (A) 9.

Q4. Consider:

```
double a[2] = {20.0, 25.0}, *p, *q;
p = a; q = p + 1;
printf("%d,%d", (int)(q-p), (int)(*q-*p));
```

What does the program print?

✓ (A) 1, 5

(B) 8, 5

(C) 1, 45

(D) 8, 45

Answer: (A) — p points to a[0] (20.0). q = p+1 points to a[1] (25.0). q - p: pointer subtraction gives the ELEMENT count between them, NOT bytes. Since both are double*, q-p = 1 (one element apart). Cast to int = 1. *q - *p = 25.0 - 20.0 = 5.0. Cast to int = 5. Output: 1,5. TRAP: many students say 8 (byte offset). Pointer subtraction gives element count!

Q5. Consider the following C program:

```
int arr[5] = {30,50,10,0,0};  
int *ptr;  
ptr = &arr[0] + 1;  
int a = *ptr;  
(*ptr)++;  
ptr++;  
printf("%d", a + (*ptr) + arr[1]);
```

What is the output?

- (A) 100
- (B) 101
- ✓ (C) 151
- (D) 111

Answer: (C) — Initial: arr = {30,50,10,0,0}. ptr = &arr[0]+1 = &arr[1], so *ptr = arr[1] = 50. a = *ptr = 50. (*ptr)++ → arr[1] becomes 51. arr = {30,51,10,0,0}. ptr++ → ptr now points to arr[2], *ptr = 10. arr[1] is now 51 (modified). printf: a + (*ptr) + arr[1] = 50 + 10 + 51 = 111. Wait — re-check: a=50 (captured before increment), *ptr after ptr++ = arr[2]=10, arr[1]=51. 50+10+51 = 111. BUT arr was modified: arr[1]=51 after increment. Actually 50+10+51=111. Hmm — check options. a=50, *ptr (arr[2])=10, arr[1]=51. Sum=111. Answer: (D) 111.

Pointer Declarations

Declaration	Type / Meaning
<code>int *p</code>	Pointer to int
<code>int **p</code>	Pointer to pointer to int
<code>int *a[5]</code>	Array of 5 pointers to int
<code>int (*a)[5]</code>	Pointer to array of 5 ints
<code>int (*fp)(int,int)</code>	Pointer to function <code>int f(int,int)</code>
<code>int *(*fp)(int)</code>	Pointer to function returning <code>int*</code>
<code>int (*a[5])(void)</code>	Array of 5 function pointers (<code>void → int</code>)
<code>const int *p</code>	Pointer to const int (data is read-only)
<code>int * const p</code>	Const pointer to int (pointer is read-only)
<code>const int * const p</code>	Const pointer to const int (both read-only)

Arithmetic Quick-Reference

Expression	What it means	Size change (int*)
<code>p + n</code>	Address n elements FORWARD	<code>+n*sizeof(*p)</code>
<code>p - n</code>	Address n elements BACKWARD	<code>-n*sizeof(*p)</code>
<code>p - q</code>	Number of elements between p and q	<code>ptrdiff_t</code> result
<code>*p</code>	Value at address p	—
<code>p[i]</code>	Identical to <code>*(p+i)</code>	—
<code>p++</code>	Move to next element, return old p	<code>+sizeof(*p)</code>
<code>++p</code>	Move to next element, return new p	<code>+sizeof(*p)</code>

DMA Safety Rules

- Always check `malloc/new` return value for `NULL` / `std::bad_alloc`.
- Always `free/delete` exactly once — double-free is undefined behaviour.
- Use `delete[]` for arrays allocated with `new[]`, never plain `delete`.
- Set pointer to `nullptr` immediately after freeing — prevents use-after-free.
- Prefer `std::vector` / smart pointers in C++ — manual heap = bugs.
- Detect leaks with: `valgrind --leak-check=full` or `-fsanitize=address`.
- Never return address of a local (stack) variable — instant dangling pointer.
- In C++, prefer `new(nothrow)` in performance-critical code to avoid exception overhead.

■ *Day 1 Build Tasks: (1) Reverse array using only pointer arithmetic. (2) Dynamic array struct with `new/delete`. (3) Compare `swap-by-pointer` vs `swap-by-reference` assembly output (godbolt.org). (4) Implement a function-pointer callback task queue — lay the foundation for your thread pool.*

Project Link: Every concept above directly applies to the C++ crawler — pointer arithmetic for buffer parsing, function pointers for thread pool callbacks, dynamic allocation for URL strings and HTML buffers, void for generic task data.*